

Patih

A Zero-Budget Retrieval-Augmented Generation System
for the Indonesian Ministry of Social Affairs Regulatory Corpus

Theory, Architecture, Implementation

Project documentation

D:\Research\Project Data\DONE locally\patih

Document version: 2026-05-19

Abstract

Patih is a question-answering assistant grounded on the Indonesian Ministry of Social Affairs (*Kementerian Sosial*, abbreviated *Kemensos*) regulatory corpus. Its Phase 1 minimum viable product (MVP) is scoped to a single regulation, *Permensos No. 8 Tahun 2023* concerning Trafficking in Persons (*Tindak Pidana Perdagangan Orang*, TPPO) and problematic Indonesian Migrant Workers (*Pekerja Migran Indonesia Bermasalah*, PMI Bermasalah). The system is built as a Retrieval-Augmented Generation (RAG) pipeline that combines a parent-document chunking scheme, hybrid sparse–dense retrieval with Reciprocal Rank Fusion, a four-tier free-tier LLM fallback chain, and a layered hallucination defence based on prompt discipline, citation whitelisting, and HalluGraph-style entity and relation grounding. This document explains the system end-to-end in order: what it does, the *theories* it relies on (twelve sub-areas, each cited and explained), the data flow, the layer-by-layer implementation, the bilingual handling, the evaluation methodology, the deployment topology, and an annotated bibliography of every paper that shaped a design decision.

Contents

1	Overview and Goals	2
1.1	Phase scope	3
1.2	Absolute constraints	3
2	Glossary of Domain and Technical Terms	3
2.1	Indonesian regulatory and legal vocabulary	3
2.2	Machine-learning and information-retrieval vocabulary	4
3	Theoretical Foundations	6
3.1	Why RAG, not long-context stuffing or fine-tuning	6
3.2	Retrieval is the ceiling, not the LLM	6
3.3	Hybrid retrieval and Reciprocal Rank Fusion	7
3.4	Parent-Document Retrieval	8
3.5	Embedder selection for Indonesian	8
3.6	LLM selection – and the limits of small Indonesian LLMs	9
3.7	Citation generation theory	10
3.8	Hallucination defence in depth	11
3.9	Why no agentic loop in Phase 1	12
3.10	GraphRAG for multi-document legal reasoning (Phase 2)	13
3.11	Cross-lingual retrieval (bilingual ID–EN)	14

3.12	Evaluation theory	15
3.13	Production engineering theory	15
4	Mathematical Summary	17
5	End-to-End Data Flow	19
5.1	Offline ingestion pipeline (run once per document)	19
5.2	Online query pipeline (per user request)	19
6	Layer-by-Layer Architecture	20
6.1	Layer 1 – Ingestion	20
6.2	Layer 2 – Retrieval	20
6.3	Layer 3 – LLM Gateway	21
6.4	Layer 4 – Validators (Hallucination Defence in Depth)	21
6.5	Layer 5 – Conversational UI	22
6.6	Layer 6 – Evaluation	22
6.7	Layer 7 – Observability	23
7	Bilingual Indonesian–English Handling	23
8	Deployment Topology	23
9	Phase 1 Build Order (chronological)	23
10	Evaluation Trajectory (v1–v6)	24
11	Roadmap	25
12	Trade-offs Explicitly Accepted in Phase 1	25
13	Open Decision Points	26
14	Repository Layout	26
15	Disclaimer	26
A	Annotated Bibliography	27
A.1	Retrieval architecture	27
A.2	Embedder and language-model selection	27
A.3	Hallucination defence and citation discipline	28
A.4	Agentic and GraphRAG (Phase 2 / Phase 3)	29
A.5	Evaluation, observability and production engineering	29
A.6	Cross-lingual retrieval	30
A.7	Background and adjacent benchmarks	30
A.8	Patih internal documents	31

1. Overview and Goals

Patih is a *Retrieval-Augmented Generation* chatbot designed to answer natural-language questions about Indonesian social-welfare regulations with *traceable, verifiable article-level citations*. The user types a question in Indonesian or English; the system retrieves the relevant articles (*Pasal*) from a parsed and indexed PDF corpus and prompts a Large Language Model (LLM) to compose

an answer that must explicitly cite the source article (e.g. “Pasal 5 ayat (2) huruf b”). Every answer is followed by a click-through citation card pointing back to the original PDF.

The name *Patih* alludes to the senior advisor in classical Javanese courts (e.g. Patih Gajah Mada): an “expert assistant who helps you understand regulation”.

1.1 Phase scope

- **Phase 1 (current MVP).** Single document: *Permensos No. 8 Tahun 2023* (13 pages, roughly 30 articles). One PDF ingested, 69 child chunks, 34 parent chunks indexed.
- **Phase 2.** Multi-document corpus (target 20–50 documents): UU 21/2007 (TPPO law), UU 18/2017 (PMI law), PP 59/2021, and other Permensos. Knowledge-Graph augmentation and inter-regulation cross references.
- **Phase 3.** Agentic multi-step reasoning, full human-in-the-loop (HITL) review dashboard, A/B testing with legal experts, migration to a paid LLM tier.

1.2 Absolute constraints

- **Zero monetary cost** – free-tier APIs and free/open-source software (FOSS) only.
- **Throughput** – roughly 100 active users per day, spread across an 8-hour working window. This is *not* 100 simultaneous requests per second.
- **Accuracy** – $\geq 90\%$ citation accuracy target, with human-in-the-loop review on flagged answers.
- **Bilingual** – Indonesian–English input and output, but article citations are always preserved verbatim in the original Indonesian.

2. Glossary of Domain and Technical Terms

Patih spans regulatory, Indonesian-legal, and machine-learning terminology. Before describing the architecture, we list the terms a reader needs to follow the rest of this document.

2.1 Indonesian regulatory and legal vocabulary

Kemensos Kementerian Sosial Republik Indonesia – the Ministry of Social Affairs of the Republic of Indonesia.

Permensos

Peraturan Menteri Sosial – ministerial regulation issued by the Minister of Social Affairs.

UU

Undang-Undang – statute / Act of Parliament.

PP

Peraturan Pemerintah – Government Regulation, hierarchically below an UU.

Perpres / Perbup

Peraturan Presiden / Peraturan Bupati – Presidential / Regent Regulation.

JDIH

Jaringan Dokumentasi dan Informasi Hukum – the national documentation network publishing Indonesian regulations.

BAB

Chapter of a regulation (e.g. *BAB I – Ketentuan Umum*).

Bagian

Section of a chapter.

Pasal	Article – the canonical citeable unit of an Indonesian regulation.
Ayat	Paragraph of an article, typeset in parentheses as “(1), (2), ...”.
Huruf	Lettered sub-item of an ayat (“a., b., c., ...”).
Angka	Numbered sub-item, usually inside <i>Ketentuan Umum</i> (Article 1 definitions).
Ketentuan Umum	“General Provisions” – the opening chapter where every legal term used in the regulation is defined.
TPPO	<i>Tindak Pidana Perdagangan Orang</i> – the criminal offence of human trafficking.
PMI Bermasalah	<i>Pekerja Migran Indonesia Bermasalah</i> – Indonesian migrant workers in distress.
Rehabilitasi Sosial	Social rehabilitation – one of the legal services provided to TPPO victims.
PDP	<i>Pelindungan Data Pribadi</i> – Indonesia’s Personal Data Protection Act, Law 27 of 2022.

2.2 Machine-learning and information-retrieval vocabulary

LLM	Large Language Model.
RAG	Retrieval-Augmented Generation – a pattern in which an LLM is grounded on retrieved external passages instead of relying on parametric memory alone.
Embedding	A dense vector representation of a text fragment, such that semantically similar fragments lie close in vector space.
Dense retrieval	Retrieval performed by encoding both query and corpus into embeddings and ranking by cosine similarity.
Sparse retrieval	Lexical / bag-of-words retrieval (e.g. BM25) operating on token statistics rather than learned embeddings.
BM25	<i>Best Matching 25</i> – a probabilistic ranking function (Robertson & Zaragoza, 2009).
Hybrid retrieval	Combining sparse and dense rankers, typically to recover the strengths of both.
RRF	Reciprocal Rank Fusion (Cormack et al., 2009) – a rank-aggregation method that combines two ranked lists by summing the reciprocal ranks $\sum_i 1/(k + \text{rank}_i)$, $k = 60$.
Parent-Document Retrieval	A retrieval pattern where a small unit (the <i>child</i>) is used for ranking but a larger unit containing it (the <i>parent</i>) is sent to the LLM for context.
Cross-encoder reranker	A two-tower neural model that jointly encodes (query, passage) pairs to produce a fine-grained relevance score. <i>Not used in Phase 1</i> due to GPU cost.

ONNX / INT8

The Open Neural Network Exchange runtime; INT8 refers to 8-bit integer post-training quantisation, used to shrink the `multilingual-e5-large` embedder from ~ 2.3 GB to ~ 600 MB for CPU-only deployment.

Lost-in-the-Middle

A documented positional bias in long-context LLMs (Liu et al., 2023): information in the middle of the context window is recalled significantly worse than at start or end (U-shaped curve). Foundational for chunking choices.

DRM

Document-Level Retrieval Mismatch (Reuter et al., 2025): the retriever selects passages from the wrong source document in corpora with many structurally similar documents.

SAC

Summary-Augmented Chunking (Reuter et al., 2025): prepend a document-level summary to each chunk to inject global context lost by chunking.

Knowledge Graph (KG)

A structured graph (V, E) of entities and typed relations – planned for Phase 2.

GraphRAG

LLM-driven KG construction + community detection + map-reduce QA (Edge et al., 2024).

HalluGraph

A KG-aligned hallucination-detection framework (Noël et al., 2025) with Entity-Grounding (EG) and Relation-Preservation (RP) scorers. AUC = 0.979 on structured legal documents.

Self-RAG Self-reflective RAG with retrieve/critique tokens (Asai et al., 2023).

CRAG

Corrective RAG: retrieval-evaluator gates fall-back search (Yan et al., 2024; open-source reproduction Yalavarthi, 2026).

Agentic RAG

Multi-step retrieval-reasoning loop modelled as a finite-horizon partially observable MDP (Mishra et al., 2026).

HITL

Human-in-the-loop – routing low-confidence answers to a human reviewer.

RAGAS

An LLM-as-judge evaluation framework for RAG (Es et al., 2023). Reference-free metrics: Faithfulness, Answer Relevancy, Context Precision/Recall.

Faithfulness

Fraction of factual claims in the answer entailed by the retrieved context.

Answer Relevancy

Semantic alignment between the user query and the produced answer.

Calibrated abstention

First-class metric measuring whether the model refrains from answering when it lacks evidence (Zhang et al., 2025).

Langfuse

An open-source LLM observability platform.

LiteLLM

An OpenAI-compatible router that fronts multiple LLM providers and implements fallback chains.

Chainlit

A Python framework for conversational UIs with streaming responses and citation cards.

3. Theoretical Foundations

This is the core of the document. Patih did not arise from intuition alone – every architectural decision is backed by a published result or an established design pattern. The twelve sub-sections below list, in build-order, the theories that shaped Patih, and for each we explain *how the theory was operationalised in code*. The detailed annotated bibliography at the end of the document (Section A) connects every citation to the precise piece of the system it informed.

3.1 Why RAG, not long-context stuffing or fine-tuning

Three architectural alternatives were considered before settling on RAG:

Fine-tuning the LLM on the corpus. Rejected. A 13-page Permensos is $\sim 5\text{--}8\text{k}$ unique tokens – far below the scale at which fine-tuning is viable. Fine-tuning at this size leads to overfitting and catastrophic forgetting, and is only worthwhile at the 10k+ document scale or for highly idiosyncratic task formats (Reuter et al., 2025).

Long-context stuffing. Feasible but rejected for Phase 1 *growth*. The Databricks long-context RAG study (Leng et al., 2024) and the foundational *Lost-in-the-Middle* result (Liu et al., 2023) together show that:

- Below $\sim 30\text{k}$ tokens, long-context stuffing can be *more* accurate than top- k RAG, because RAG miss-rates exceed long-context positional-bias loss.
- Above $\sim 32\text{k}$ tokens, accuracy drops 10–20% when the relevant information lies in the middle of the context (the U-shaped recall curve).
- Only a handful of state-of-the-art LLMs sustain consistent accuracy above 64k tokens.

The Liu et al. U-curve can be stated compactly: letting $\text{acc}(p, L)$ be the probability that an LLM correctly recalls a fact placed at relative position $p \in [0, 1]$ inside a context of total length L tokens, the empirical fit is roughly

$$\text{acc}(p, L) \approx \alpha(L) - \beta(L)p(1-p), \quad (\text{LM})$$

with the two endpoints maximised, the middle ($p = 0.5$) minimised, and the gap $\beta(L)$ growing with L . Practically: a fact in the middle of an 80k-token context can be recalled $\sim 20\text{ pp}$ less reliably than the same fact at the head. Patih’s parent-document chunking sidesteps this entirely by ensuring the LLM only ever sees the small handful of Pasal that the retriever has surfaced.

Since Patih is built to *scale to the multi-document corpus in Phase 2*, RAG with structure-aware chunking is the right substrate from the start: it does not regress on small corpora and it does not break when the corpus grows past the long-context regime.

RAG with structure-aware chunking. Selected. The chosen pattern is hybrid sparse+dense retrieval with parent-document expansion, an explicit cross-reference resolver and an always-on definition injector. The remaining sub-sections explain each of these decisions.

3.2 Retrieval is the ceiling, not the LLM

The single most important design principle, drawn from *Legal-RAG-Bench* (Butler & Butler, 2026): for legal RAG systems the *retrieval* stage dominates correctness more than the generator does. In their experiments, swapping the embedder yielded a +17.5 percentage-point gain in correctness, while swapping the LLM gave only a moderate improvement. The authors further observed that many errors attributed to hallucination are in fact triggered by retrieval failures.

Patih therefore concentrates engineering effort on chunking, hybrid retrieval, parent expansion, cross-reference resolution and an always-on definition injector *before* considering a model upgrade.

This was empirically re-confirmed inside Patih’s own evaluation trajectory (Section 10): the largest jumps in citation accuracy came from increasing `top_k_fused` from 8 to 12 and from sharpening retrieval-side prompt rules, not from changing models.

3.3 Hybrid retrieval and Reciprocal Rank Fusion

Sparse BM25 is strong on rare, exact technical terms (“TPPO”, “Rehabilitasi Sosial”), while dense embeddings are strong on paraphrase and semantic similarity (“what are the victim’s rights?” matching “korban berhak memperoleh ...”). Combining them via Reciprocal Rank Fusion (Cormack et al., 2009) is a well-established technique that consistently outperforms either component alone on heterogeneous corpora.

BM25 – the sparse scorer. For a query $q = \{t_1, \dots, t_m\}$ and a document d , BM25 ranks d by

$$\text{BM25}(q, d) = \sum_{t \in q \cap d} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{|\bar{d}|} \right)}, \quad (1)$$

where $f(t, d)$ is the term frequency of t in d , $|d|$ is the document length in tokens, $|\bar{d}|$ is the average document length in the corpus, and the inverse document frequency is

$$\text{IDF}(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}, \quad (2)$$

with N the corpus size and $n(t)$ the number of documents containing t . Patih uses the canonical defaults $k_1 = 1.5$, $b = 0.75$ (Robertson & Zaragoza, 2009).

Dense retrieval – the semantic scorer. Let $\phi : \text{text} \rightarrow \mathbb{R}^D$ be the embedder (for `multilingual-e5-large`, $D = 1024$). For query q and chunk c the dense score is the cosine similarity of their embeddings,

$$\text{score}_{\text{dense}}(q, c) = \cos(\phi(q), \phi(c)) = \frac{\phi(q)^\top \phi(c)}{\|\phi(q)\|_2 \|\phi(c)\|_2} \in [-1, 1]. \quad (3)$$

The e5 family requires its asymmetric prefix convention: $\phi(q) = \text{Enc}(\text{"query: "} \| q)$ and $\phi(c) = \text{Enc}(\text{"passage: "} \| c)$, where $\|$ denotes string concatenation. Forgetting this prefix degrades MIRACL-id nDCG@10 by 5–10 points and is a common bug.

Reciprocal Rank Fusion. BM25 scores and cosine scores live in different magnitudes and are not directly comparable. RRF combines them *at the rank level* (Cormack et al., 2009): for a chunk c appearing at rank $r_i(c) \in \{1, 2, \dots\}$ in list i (or $r_i(c) = \infty$ if absent),

$$\text{RRF}(c) = \sum_{i \in \{\text{dense}, \text{sparse}\}} \frac{1}{k + r_i(c)}, \quad k = 60. \quad (4)$$

The constant $k = 60$ damps the influence of low-rank items: rank 1 contributes $1/61 \approx 0.0164$, rank 10 contributes $1/70 \approx 0.0143$ – a 13 % gap, large enough to favour the top-ranked items but small enough that a chunk in the top-5 of both lists beats a chunk that is rank-1 in only one. Patih uses dense top-15 and sparse top-15, fused to top-12 (raised from the original top-8 after Phase 1 tuning – Section 10).

HyPA-RAG analog. The same hybrid + RRF combination is the workhorse of the closest published analog to Patih, HyPA-RAG for NYC Local Law 144 (Kalra et al., 2025), which integrates a query-complexity classifier, hybrid dense+sparse+KG retrieval, and a tailored evaluation framework. HyPA-RAG is Patih’s Phase 2 reference design.

3.4 Parent-Document Retrieval

A small *child* unit is the right granularity for matching a focused query, but the same small unit is the *wrong* granularity for the LLM, which needs surrounding context to compose a faithful answer. The Parent-Document pattern (popularised by LangChain and LlamaIndex) decouples the two: rank on children, expand to parents for generation.

For Indonesian regulatory text this maps naturally to the hierarchy BAB → Bagian → Pasal → ayat → huruf:

- **Child chunk (retrieval unit)** = one ayat, or one huruf when the ayat is long.
- **Parent chunk (generation unit)** = the entire Pasal.

The closest published analog is *Tree-Based Text Retrieval via Hierarchical Clustering* applied to Taiwanese regulations (Yu & Tsai, 2025), which exploits the BAB/Pasal hierarchy directly and outperforms naive flat top-*k* retrieval; their key benefit – adaptively selecting the right granularity without a fixed *k* – inspired Patih’s parent-expand-then-cross-ref pattern.

Document-Level Retrieval Mismatch and Summary-Augmented Chunking. For Phase 2 (multi-document) Patih will additionally adopt *Summary-Augmented Chunking* (SAC) from Reuter et al. (2025), which prepends a document-level synthetic summary to each chunk to mitigate Document-Level Retrieval Mismatch (DRM) – the failure mode in which the retriever picks the right Pasal from the *wrong* regulation. SAC is unnecessary in Phase 1 (single document) but is on the explicit Phase 2 roadmap.

Always-on definition injection. Almost every Indonesian regulation defines its key terms in Pasal 1 (*Ketentuan Umum*). Patih always prepends Pasal 1 to the LLM context, irrespective of retrieval scores. This is a deterministic counterpart to the cross-reference resolver and dramatically improves definition-style queries.

Cross-reference resolution. Indonesian legal text is dense with phrases such as “sebagaimana dimaksud dalam Pasal 2 ayat (2)”. A regex resolver (`Pasal\s+(\d+)\s+ayat\s+((\d+)\s+)`) on every retrieved parent pulls referenced Pasal into context. In Phase 2 this Layer-1 resolver will be superseded by a knowledge-graph traversal (Section 3.10).

3.5 Embedder selection for Indonesian

The embedder choice was made on numbers, not vibes. The community-maintained *LazarusNLP Indonesian Sentence Embeddings* benchmark tracks Indonesian retrieval performance on MIRACL-id, TyDiQA-id and IndoSTSB. The top entry as of 2026:

Model	Params	R@1 MIRACL-id	nDCG@10 MIRACL-id	R@1 TyDiQA-id
multilingual-e5-large	560M	70.83%	76.16%	94.69%
multilingual-e5-base	278M	68%	74%	93.45%
multilingual-e5-small	118M	70.20%	74.80%	92%
all-Indo-e5 Small-v4	118M	68.33%	73.23%	90%
all-NusaBERT Large-v4	337M	65%	71%	93.62%

BGE-M3 as an alternative. BGE-M3 (Chen et al., 2024) is a unified dense + sparse + multi-vector embedder; it cuts the BM25 dependency but is slightly weaker on Indonesian-only retrieval. Patih considered and *rejected* BGE-M3 in favour of multilingual-e5-large for Phase 1, with the option to revisit if hybrid orchestration simplification becomes valuable.

Why not a legal-Indonesian-specific embedder? There is no published production-grade analog of *LEGAL-BERT*, *HUKUKBERT* (Türkçe legal; Öztürk et al., 2026), or the Indian-law BERT family (Paul et al., 2023; Joshi et al., 2024 IL-TUR) for Indonesian as of 2026. This is a noted gap; brief v2 flags fine-tuning a legal-Indonesian embedder from *multilingual-e5-large* as a 4–8-week open research project. Patih runs without it.

Cost-driven substitution. In the current deployment, disk-constraints have forced Patih to use the Gemini embedding API in place of a local INT8 ONNX e5-large. The architecture is unchanged; only the embedding endpoint differs, and the upgrade path is documented in `deploy/scripts/quantize_e5.py`.

INT8 post-training quantisation. Self-hosted deployment uses 8-bit integer quantisation of the FP32 weight tensors. For a weight tensor $W \in \mathbb{R}^{m \times n}$ with observed range $[W_{\min}, W_{\max}]$, the symmetric scheme picks scale s and zero-point z as

$$s = \frac{W_{\max} - W_{\min}}{2^8 - 1}, \quad z = \text{round}\left(-\frac{W_{\min}}{s}\right), \quad (5)$$

and quantises each entry by

$$W_{ij}^{\text{int8}} = \text{clip}\left(\text{round}\left(\frac{W_{ij}}{s}\right) + z, -128, 127\right). \quad (6)$$

Dequantisation at inference time is $\widetilde{W}_{ij} = s(W_{ij}^{\text{int8}} - z)$. The memory footprint of *multilingual-e5-large* shrinks from

$$560 \text{ M params} \times 4 \text{ bytes (FP32)} \approx 2.24 \text{ GB} \quad \longrightarrow \quad 560 \text{ M params} \times 1 \text{ byte (INT8)} \approx 0.56 \text{ GB},$$

with a typical 1.5–2× throughput improvement on CPU and an empirical $\leq 1\%$ nDCG@10 degradation on MIRACL-id – acceptable for Phase 1.

3.6 LLM selection – and the limits of small Indonesian LLMs

The default LLM is selected on two empirical points and one constraint.

Empirical point 1: Small Indonesian-targeted LLMs underperform on professional reasoning. Koto et al. (2024), *Cracking the Code*, evaluated 27 LLMs (including Indonesian-targeted Merak, Komodo, Cendol, and earlier SEA-LION) on Indonesian professional exams and found that the small ID-targeted models performed *near-random* on legal and professional reasoning. Only frontier models (GPT-4o, Claude Opus, Gemini 1.5 Pro at the time of writing; their successors today) achieved meaningful accuracy. This rules out a self-hosted small-model default for Patih.

Empirical point 2: SEA-HELM tracks the current open-model leaders. The Southeast-Asian Holistic Evaluation of Language Models leaderboard (Susanto et al., 2025) shows Qwen 3 family models consistently leading on SEA tasks (e.g. Qwen 3 VL 32B at 68.41 SEA-HELM Indonesian). Patih’s Tier-3 fallback is Qwen-family on Cerebras precisely because of this signal.

Constraint: zero budget rules out paid frontier models in Phase 1. The absolute-best citation discipline in 2026 belongs to Claude Sonnet 4.6, but it is paid-only. The Phase 1 fallback chain therefore picks free-tier providers in descending order of empirical legal-citation discipline: Gemini 2.5 Flash → Groq Llama 3.3 70B → Cerebras Qwen 3 32B / Llama 3.1 8B → OpenRouter DeepSeek R1.

Why Sonnet 4.6 remains the migration target. The Stanford HAI legal-tools study (Magesh et al., 2024) shows that even premium commercial tools (Westlaw AI-Assisted Research, Lexis+ AI) hallucinate 17–34 % of the time. Free-tier models cannot beat this; brief v2 estimates raw citation imprecision at 25–45 % before Layer 2 validators (Section 3.8). The pragmatic ceiling for the Phase 1 system is set by this empirical fact, not by our wishful thinking.

Decoding configuration. Patih asks the LLM for grounded, low-creativity output. For a vocabulary V and logits $\mathbf{z} \in \mathbb{R}^{|V|}$ produced by the model at decoding step t , the next-token distribution is the temperature-softmax

$$P_T(t_i | t_{<i}) = \frac{\exp(z_i/T)}{\sum_{j=1}^{|V|} \exp(z_j/T)}, \quad (7)$$

where the temperature $T \in (0, \infty)$ controls sharpness: $T \rightarrow 0$ reduces to greedy decoding (argmax), $T = 1$ recovers the model’s native distribution, and $T > 1$ flattens it. Patih sets $T = 0.1$ for the generator call – near-greedy decoding favours verbatim Pasal quotation over creative paraphrase, which is exactly what legal citation discipline requires.

3.7 Citation generation theory

Citation accuracy is its own research problem, not a side-effect of answer accuracy.

Qian et al. 2024 – “On the Capacity of Citation Generation by LLMs”. A systematic analysis of seven LLMs on WebGLM-QA / ASQA / ELI5: existing eval metrics over-penalise unnecessary or excessive citations, and the authors propose a *Generate-then-Refine* procedure to add missing relevant citations and prune irrelevant ones *without altering the response text*. Patih’s evaluation framework adopts this principle by reporting micro-recall *and* precision separately (Section 6.6); “over-citations” in Patih’s v6 run hurt precision but not recall, exactly as Qian et al. predicted.

Citation metrics, formally. Let $C_{\text{gt}}^{(i)}$ be the ground-truth Pasal set for question i in a test set of size N , and $C_{\text{model}}^{(i)}$ the set of Pasal citations extracted from the model’s response. Per-query precision, recall, and F_1 are

$$P^{(i)} = \frac{|C_{\text{model}}^{(i)} \cap C_{\text{gt}}^{(i)}|}{|C_{\text{model}}^{(i)}|}, \quad R^{(i)} = \frac{|C_{\text{model}}^{(i)} \cap C_{\text{gt}}^{(i)}|}{|C_{\text{gt}}^{(i)}|}, \quad F_1^{(i)} = \frac{2 P^{(i)} R^{(i)}}{P^{(i)} + R^{(i)}}. \quad (8)$$

Patih’s headline “Citation Accuracy” metric is the *micro-recall* – the count-weighted aggregate that does not let easy single-citation questions dominate hard multi-citation ones,

$$R_{\text{micro}} = \frac{\sum_{i=1}^N |C_{\text{model}}^{(i)} \cap C_{\text{gt}}^{(i)}|}{\sum_{i=1}^N |C_{\text{gt}}^{(i)}|}. \quad (9)$$

A complementary set-overlap metric is the mean Jaccard index,

$$J = \frac{1}{N} \sum_{i=1}^N \frac{|C_{\text{model}}^{(i)} \cap C_{\text{gt}}^{(i)}|}{|C_{\text{model}}^{(i)} \cup C_{\text{gt}}^{(i)}|}. \quad (10)$$

The v6 evaluation gave $R_{\text{micro}} = 0.787$, $J = 0.823$, and precision ≈ 0.75 – the gap between the two confirmed Qian et al.’s observation that micro-recall is robust to over-citation, whereas precision is not.

Calibrated abstention as a separate axis. Zhang et al. define *refusal precision* as a third independent dimension. Treating refusals as positive-class predictions,

$$P_{\text{refusal}} = \frac{\text{TP}_{\text{refusal}}}{\text{TP}_{\text{refusal}} + \text{FP}_{\text{refusal}}}, \quad (11)$$

where $\text{TP}_{\text{refusal}}$ is the number of Tier-4/5 questions that the system correctly refused, and $\text{FP}_{\text{refusal}}$ is the number of Tier-1/2/3 questions that the system wrongly refused. Patih’s v6 evaluation gave $P_{\text{refusal}} = 1.00$ (10/10).

VeriCite (Qian et al., 2025). The follow-up paper proposes a three-stage pipeline: initial answer generation $\rightarrow$ NLI-based claim verification $\rightarrow$ supporting-evidence selection $\rightarrow$ final answer refinement. Patih’s Layer 2 validator chain (citation extractor $\rightarrow$ whitelist $\rightarrow$ EG $\rightarrow$ RP) is a simplified, regex-and-set-based variant of VeriCite. Phase 3 plans to replace the regex EG/RP with NLI-based verification per VeriCite.

Calibrated abstention (Zhang et al., 2025). For 3-ply case-based legal arguments, Zhang et al. define an automated pipeline that scores *faithfulness* (no fabricated factors), *factor utilisation* (use the right factors) and *abstention* (refuse when no factual basis exists) as three independent dimensions. Crucially, they find that most LLMs fail the abstention test: they generate spurious arguments even when instructed to stop. Patih treats refusal precision as a first-class metric (Tier 4 + Tier 5 of the eval set, Section 6.6), and the system prompt mandates the exact refusal sentence “*Informasi tersebut tidak diatur secara spesifik dalam ...*”. v6 evaluation showed 100 % refusal precision (10 / 10).

3.8 Hallucination defence in depth

A single defence is insufficient; defence in depth is what makes free-tier LLM legal QA acceptable. The literature picture:

- **Magesh et al. 2024 (Stanford HAI).** 17–34 % hallucination on premium commercial tools (Westlaw, Lexis+, Practical Law / Ask GPT). Lexis+’s “hallucination-free” marketing claim was empirically falsified.
- **Noël et al. 2025 (HalluGraph).** A KG-aligned detector decomposed into *Entity Grounding* (EG: response entities $\subseteq$ context entities) and *Relation Preservation* (RP: asserted relations supported by context). AUC = 0.979 on long structured legal documents, AUC ≈ 0.89 on harder generative tasks; consistently beats semantic-similarity baselines, which the authors show *tolerate entity substitutions* (a dangerous failure mode for legal text where “Pasal 5” and “Pasal 6” embed close together).
- **Sardana 2025 – “Who Detects Hallucinations Best?”.** Benchmarks LLM-as-Judge, Prometheus, Lynx, HHEM and TLM across six RAG applications. No single detector wins; an *ensemble* of two or three is more reliable. Patih’s four-validator pipeline (citation extractor, whitelist, EG, RP) is a deliberately heterogeneous ensemble.
- **Liu et al. 2026 – “Fast and Faithful”.** A production-scale real-time verifier with adaptive inference, balancing latency and verification coverage. The Phase 3 candidate for replacing Patih’s Phase 1 regex EG/RP.

Entity Grounding and Relation Preservation, formally. Let $\mathcal{E}(\cdot)$ extract the set of legal entities from a text (Pasal references, defined terms, named institutions) and let $\mathcal{R}(\cdot)$ extract the set of asserted relations as (subject, predicate, object) triples. Given a response r and a retrieved context ctx , the HalluGraph scorers are

$$\text{EG}(r, \text{ctx}) = \frac{|\mathcal{E}(r) \cap \mathcal{E}(\text{ctx})|}{|\mathcal{E}(r)|} \in [0, 1], \quad (12)$$

$$\text{RP}(r, \text{ctx}) = \frac{|\mathcal{R}(r) \cap \mathcal{R}(\text{ctx})|}{|\mathcal{R}(r)|} \in [0, 1]. \quad (13)$$

EG penalises hallucinated entities (e.g. “Pasal 99” when the document has only Pasal 1–34); RP penalises invented relationships (e.g. “Menteri menetapkan sanksi pidana” when the context establishes only administrative sanctions). Patih flags a response for HITL review when $\text{EG} < 0.90$ or $\text{RP} < 0.80$.

Discriminator quality (AUC). The Area Under the Receiver Operating Characteristic Curve summarises how well a scorer separates faithful from hallucinated responses across all possible thresholds:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(u)) du = \Pr[\text{score}(x^+) > \text{score}(x^-)], \quad (14)$$

where x^+ is a faithful response and x^- a hallucinated one drawn from the held-out set. Noël et al. (2025) report $\text{AUC} = 0.979$ for HalluGraph on long structured legal documents and $\text{AUC} \approx 0.89$ on shorter generative tasks, beating embedding-similarity baselines that tolerate entity substitution.

Composition of defences (independence model). If a hallucinated response has probability p_0 before any defence and each layer $\ell \in \{1, \dots, L\}$ catches a fraction $c_\ell \in [0, 1]$ of those that reach it, the residual rate after the full stack is (under the independence assumption)

$$p_L = p_0 \cdot \prod_{\ell=1}^L (1 - c_\ell). \quad (15)$$

With the literature-derived numbers $p_0 \in [0.25, 0.45]$ and approximate catch rates $c_{\text{prompt}} \approx 0.4$, $c_{\text{whitelist}} \approx 0.5$, $c_{\text{EG/RP}} \approx 0.5$, $c_{\text{HITL}} \approx 0.7$, the composition gives $p_L \in [0.011, 0.020]$, consistent with the empirical $\leq 5\%$ end-state target. The table below makes the residual at each stage explicit:

Stage	Catch rate c_ℓ (assumed)	Expected residual imprecision
Raw LLM output	–	25–45 %
+ Strict prompting	0.40	15–27 %
+ Citation whitelist	0.50	7.5–13.5 %
+ HalluGraph EG / RP scorers	0.50	3.8–6.8 %
+ HITL on the $\sim 15\%$ flagged	0.70	$\leq 5\%$ effective

This is the realistic envelope for a zero-budget MVP, not a perfection claim. Equation (15) also makes explicit why defence-in-depth is multiplicative – adding any single new layer with $c_\ell > 0$ strictly reduces the residual, but the marginal gain shrinks as $\prod(1 - c_\ell)$ approaches zero.

3.9 Why no agentic loop in Phase 1

Agentic RAG – multi-step retrieve-reason-critique loops – is attractive on paper but dangerous in legal applications.

Self-RAG (Asai et al., 2023). The canonical pattern: a single LM emits “reflection tokens” that govern when to retrieve, what to retrieve, whether to keep the result, and when to stop. Strong gains on factuality and citation accuracy on open-domain QA.

CRAG (Yan et al., 2024). Adds a retrieval evaluator that triggers corrective actions (web search fallback) when retrieved documents are judged low-quality. The open-source reproduction by Yalavarthi (2026) shows that the T5-based evaluator relies primarily on *named-entity alignment* rather than semantic similarity – consistent with HalluGraph’s finding that entity grounding is the more discriminative signal.

Doctor-RAG (Jiao et al., 2026). A *diagnose-and-repair* framework that localises the failure point in a multi-hop trajectory and intervenes only at that point, reusing the validated reasoning prefix. Reduces token consumption substantially compared with full pipeline reruns.

Chain-of-Retrieval (Wang et al., 2025). An “o1-style” RAG model that retrieves and reasons step-by-step, dynamically reformulating the query. >10 EM-point improvement on multi-hop QA over single-shot RAG baselines.

The SoK warning. Mishra et al. (2026) formalise Agentic RAG as a finite-horizon partially observable Markov Decision Process (POMDP)

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma, H), \quad (16)$$

where $\mathcal{S}$ is the latent state (current context plus reasoning history), $\mathcal{A} = \{\text{retrieve, generate, tool-call, stop}\}$ the action space, $T(s' | s, a)$ the transition kernel, $R(s, a)$ the reward, Ω the observation space (retrieved chunks, tool outputs), $O(o | s)$ the observation function, $\gamma \in (0, 1]$ the discount factor, and H the horizon (the maximum number of hops). A policy $\pi : \Omega^* \rightarrow \mathcal{A}$ is then judged by its expected return

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^H \gamma^t R(s_t, a_t) \right]. \quad (17)$$

This formal framing makes the failure modes precise:

- *Compounding hallucination propagation* – an error in o_t degrades all downstream actions $a_{t+1}, \dots, a_H$.
- *Memory poisoning* – a corrupted history element enters the belief state and persists.
- *Retrieval misalignment* – T becomes non-stationary because the corpus index drifts.
- *Cascading tool-execution* – $\mathcal{A}$ contains side-effecting tools whose failures are not in R .

For legal QA the cost of one bad chain is much higher than for vanilla RAG. Patih therefore deliberately defers agentic loops to Phase 3, with strict cost-bounds ($H = 3$), explicit HITL fallback at the loop boundary, and per-hop logging for debugging.

3.10 GraphRAG for multi-document legal reasoning (Phase 2)

The Phase 2 leap is a knowledge graph plus a graph-aware retriever.

From Local to Global (Edge et al., 2024 / Microsoft GraphRAG). LLM-driven entity extraction $\rightarrow$ community detection (Leiden) $\rightarrow$ community summaries $\rightarrow$ map-reduce QA. Best in class for *global sensemaking* queries (“what are the dominant themes in this corpus?”) at the 1 M-token scale.

Domain-Partitioned Hybrid RAG (Goel et al., 2026). The most directly relevant analog: an Indian legal-AI system with three specialised RAG pipelines (Supreme Court case law, statutes, IPC) plus a Neo4j-backed Knowledge Graph capturing relationships among cases, statutes, judges and citations. An LLM-driven agentic orchestrator routes queries across the modules. Result: **70 % pass rate** versus 37.5 % for RAG-only baseline – nearly a $2\times$ lift. Patih’s Phase 2 architecture is modelled on this design, restricted to Indonesian regulation and PMI/TPPO scope.

Reuter et al. 2025 – the multi-document failure mode that motivates SAC. Already discussed in Section 3.4: in large legal corpora the retriever picks chunks from the wrong source document because article-1-style “Ketentuan Umum” chunks are nearly identical across regulations. SAC mitigates this.

Trade-off explicitly accepted. GraphRAG construction is non-trivial (Goel et al. estimate 2–4 engineer-weeks for the pipeline plus 1–2 weeks of legal-aware curation per 50 PDFs). Patih defers this cost to Phase 2 once the corpus exceeds the single-document scope.

KG schema for the Indonesian regulatory corpus. Brief v2 fixes the node and edge typing in advance, so that Phase 2 ingestion is deterministic where possible and LLM-driven only where required. The proposed schema is

$$V = \{\text{Regulasi, Pasal, Ayat, Huruf, Definisi, Subjek_Hukum, Sanksi, Lembaga, Prosedur}\},$$

$$E = \{\text{rujuk_ke, mendefinisikan, mengatur_subjek, dikenakan_kepada, dilaksanakan_oleh, derogasi, meler}$$

Two construction passes are anticipated. *Pass 1 – deterministic.* The existing regex parser already extracts **Regulasi**, **Pasal**, **Ayat** and **Huruf** nodes, plus every **rujuk_ke** edge via the in-text reference resolver of Section 3.4. *Pass 2 – LLM-driven.* Semantic relations such as “Pasal A *melengkapi* Pasal B because both regulate the same Subjek_Hukum” require an LLM extractor with human-in-the-loop curation, following Goel et al.’s Indian legal-AI pattern. A **Regulasi** node additionally carries **tanggal_berlaku** and **tanggal_dicabut**, so the default query scope is the time slice $\{r : \text{tanggal_berlaku} \leq \text{now} < \text{tanggal_dicabut}\}$ – a non-negotiable property for regulatory QA because answering with a repealed Pasal is a worse failure than not answering at all.

3.11 Cross-lingual retrieval (bilingual ID–EN)

Patih supports Indonesian and English queries over an Indonesian-only corpus. Three results inform the design:

Translate the *query*, not the corpus. Badenes-Olmedo et al. (2019) studied cross-lingual legal retrieval via topic hierarchies on EuroVoc synsets; the lesson generalises – translating the corpus loses domain-specific nuance (*eksploitasi*, *rehabilitasi*, *asistensi* have legal-specific meanings without stable English equivalents). Translating the query keeps the corpus monolingual Indonesian, and **multilingual-e5-large** is already trained for cross-lingual alignment.

CroSearch-R1 (Qi et al., 2026). A search-augmented RL framework that integrates multi-lingual knowledge via Group Relative Policy Optimisation (GRPO) with cross-lingual rollouts. Frontier-grade cross-lingual RAG – Patih’s Phase 3 candidate if bilingual usage becomes the dominant pattern.

The Patih pipeline. Detect $\rightarrow$ translate-EN-query-to-ID-via-Gemini $\rightarrow$ retrieve-in-ID-corpus $\rightarrow$ generate-in-original-language $\rightarrow$ preserve-Pasal-citations-verbatim-in-Indonesian. The system-prompt clause that enforces verbatim Indonesian citations is paired with a regex post-validator that asserts citations remain in Indonesian.

3.12 Evaluation theory

RAGAS (Es et al., 2023). The canonical reference-free RAG evaluation framework. Formally, given an answer a , the context ctx , and the original question q , let $\mathcal{C}(a) = \{c_1, \dots, c_{n_a}\}$ denote the set of atomic claims extracted from a by an LLM-as-judge, and let $\mathbf{1}[c \models \text{ctx}]$ indicate whether claim c is entailed by the context. The three primary metrics are:

Faithfulness – the proportion of claims supported by the context:

$$F(a, \text{ctx}) = \frac{1}{n_a} \sum_{c \in \mathcal{C}(a)} \mathbf{1}[c \models \text{ctx}] \in [0, 1]. \quad (18)$$

Answer Relevancy – the cosine similarity, in embedding space, between the original question and synthetic questions $\{q^{(1)}, \dots, q^{(m)}\}$ that the judge generates from the answer:

$$\text{AR}(q, a) = \frac{1}{m} \sum_{j=1}^m \cos(\phi(q), \phi(q^{(j)})). \quad (19)$$

Context Precision – whether the chunks in the retrieved context that are actually relevant come first. With $\text{rel}_k \in \{0, 1\}$ marking chunk k as relevant by the judge,

$$\text{CP} = \frac{1}{|\{k : \text{rel}_k = 1\}|} \sum_{k=1}^K \frac{\text{rel}_k \cdot \sum_{j=1}^k \text{rel}_j}{k}. \quad (20)$$

Patih sets the production thresholds $F \geq 0.85$ and $\text{AR} \geq 0.85$, in line with brief v2. To avoid double-burning the same provider’s daily quota, the generator and the RAGAS judge are configurable separately (`-provider` and `-judge-provider`).

Ensemble eval (Sardana 2025). Already discussed – no single hallucination detector is reliable; Patih runs multiple in parallel.

Real-time verification (Liu et al., 2026). “Fast and Faithful” shows that full-document grounding under latency constraints is feasible with adaptive inference. Phase 3 candidate for Patih’s verifier upgrade.

3.13 Production engineering theory

Vector-database choice (Pan et al., 2023). The survey of vector DBs identifies five obstacles: vagueness of semantic similarity, large vector size, comparison cost, lack of natural partitioning, and difficulty of hybrid queries. For Phase 1 (50k vectors, single tenant), Chroma embedded with SQLite backing is sufficient. Pan et al. informed the Phase 2 plan to migrate to Qdrant self-host or pgvector.

Filtered ANN (Amanbayev et al., 2026). Once Patih scopes per-regulation (“answer only from Permensos 8/2023”), filtered ANN with payload indexes (pre-filter rather than post-filter) becomes important. Phase 2 plan.

Tail-tolerant retry for LLM gateways (Tian Pan, 2026). LiteLLM’s default retry policies can add 30s+ latency on failing tails. Patih’s gateway tunes timeouts aggressively (5s per attempt, max 2 retries, total budget 15s) following this lesson.

Free-tier landscape (vendor docs accessed 2026-05-17). The four-tier fallback chain (Gemini → Groq → Cerebras → OpenRouter) is selected on RPM/RPD published limits, with aggregated headroom $\sim 70\text{--}90$ RPM – comfortably above the expected 12–15 RPM peak from 100 daily active users. The empirically observed Gemini 2.5 Flash free tier of 20 RPD (versus the previously documented 1500 RPD) forced Patih to make Groq the new primary in v5/v6 evaluation runs.

Capacity model. Let U be the number of daily active users, Q the queries per user per session, T_w the working-hours window in hours, and Λ_p the RPM ceiling of provider p . The aggregated provider capacity is

$$\Lambda = \sum_p \Lambda_p \approx 70\text{--}90 \text{ RPM}, \quad (21)$$

and the expected steady-state arrival rate (averaged across the work day) is

$$\lambda = \frac{U \cdot Q}{60 T_w} = \frac{100 \times 10}{60 \times 8} \approx 2.1 \text{ RPM}. \quad (22)$$

The utilisation $\rho = \lambda/\Lambda \approx 0.03$ is small in steady state – the system is provisioned roughly 30× over the average load. The headroom is sized for burst absorption, not average throughput.

Latency under queueing (Little’s law). For a stable queue with arrival rate λ and mean sojourn time W , the mean number in system is

$$L = \lambda W. \quad (23)$$

If B users arrive within a window of τ seconds and the aggregate capacity is Λ requests per second, the last user’s expected wait is approximately

$$W_{\text{burst}} \approx \frac{B}{\Lambda} - \tau, \quad \text{when } \frac{B}{\tau} > \Lambda. \quad (24)$$

For $B = 50$, $\tau = 60$ s, $\Lambda \approx 70/60 \approx 1.17$ req/s, this gives $W_{\text{burst}} \approx 43 - 60 = -17$ s, i.e. no queueing – 50 users within a minute is absorbed. For $B = 100$ in the same minute, $W_{\text{burst}} \approx 85 - 60 = 25$ s, which sets Patih’s P95 latency target at ≤ 30 s. Above $B \approx 120$ within 60 s the system breaks the latency budget, defining the hard burst ceiling at which the zero-budget Phase 1 architecture must migrate to paid tiers.

Pre-emptive rate limiting – token bucket. Patih does not wait for a provider to return HTTP 429 (“Too Many Requests”) before slowing down. The file `app/llm/rate_limiter.py` maintains, per provider, a *token bucket*: a counter $b_t \in [0, C]$ that refills at constant rate r tokens per second up to capacity C , and is decremented by 1 on each outgoing request. Formally,

$$b_{t+\Delta t} = \min(C, b_t + r \Delta t) - \mathbf{1}[\text{request at } t + \Delta t]. \quad (25)$$

A request is admitted iff $b_t \geq 1$; otherwise the gateway either waits $(1 - b_t)/r$ seconds or fails over to the next provider in the chain. For Gemini 2.5 Flash with the observed 15 RPM ceiling, $C = 15$, $r = 15/60 = 0.25$ tokens/s. The token bucket smooths bursts up to size C while bounding the long-run rate to r , which is exactly the constraint imposed by free-tier RPM ceilings. The pattern dates to ATM-network traffic shaping (1990s) and is the canonical algorithm for client-side rate limiting in modern API gateways.

Semantic caching (Phase 2). Indonesian Kemensos staff are expected to ask repetitive questions (“Apa definisi Korban TPPO?” is the same question whether typed as “Siapa korban TPPO?” or “Definisi korban perdagangan orang”). A semantic cache returns the previous answer when the new query is sufficiently similar to a cached one:

$$\hat{a} = a^* \quad \text{if} \quad \max_{(q^*, a^*) \in \mathcal{K}} \cos(\phi(q), \phi(q^*)) \geq \tau_{\text{cache}}, \quad (26)$$

where $\mathcal{K}$ is the cache and $\tau_{\text{cache}} \in [0.9, 0.97]$ is the similarity threshold (tighter for legal QA than for casual chat, because a small semantic shift in the question can change which Pasal applies). Open-source implementations: GPTCache (mature, drop-in into the Anthropic / OpenAI SDK) or Redis with vector search. Cache invalidation policy for regulatory text: short TTL (~ 24 h) plus mandatory flush on any document amendment – a stale answer about an amended Pasal is worse than a cache miss. Brief v2 estimates semantic-cache hit rates of 30–60 % for staff-internal use, cutting downstream LLM cost proportionally. *Not* in Phase 1 (single tenant, low volume); on the Phase 2 roadmap.

4. Mathematical Summary

Every formula introduced in Section 3 is listed below in a single reference table, in the order it appears in the document. The equation numbers are stable single-pass (each equation is tagged explicitly with `\tag`), so the table below works even on the first `pdflatex` run.

Eq.	Name and compact form	Role in Patih
(1)	BM25 score $\sum_{t \in q \cap d} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b d / d)}$	Sparse-retrieval ranker; recovers exact-term matches that dense embeddings miss (“TPPO”, “PMI Bermasalah”).
(2)	IDF weighting $\log \frac{N - n(t) + 0.5}{n(t) + 0.5}$	Down-weights frequent tokens; rare legal terms dominate the BM25 score.
(3)	Cosine similarity $\cos(\phi(q), \phi(c)) = \frac{\phi(q)^\top \phi(c)}{\ \phi(q)\ \ \phi(c)\ }$	Dense-retrieval ranker over the multilingual-e5-large embedding space.
(4)	Reciprocal Rank Fusion $\text{RRF}(c) = \sum_i \frac{1}{k + r_i(c)}, \quad k = 60$	Combines BM25 ranks and dense ranks without rescaling either; produces the top-12 fused list.
(5)	INT8 quantisation scale / zero-point $s = (W_{\max} - W_{\min})/255, \quad z = \text{round}(-W_{\min}/s)$	Calibrates the FP32 $\rightarrow$ INT8 mapping for the self-hosted embedder.
(6)	INT8 weight transform $W_{ij}^{\text{int8}} = \text{clip}(\text{round}(W_{ij}/s) + z, -128, 127)$	Shrinks e5-large from 2.24 GB to 0.56 GB for CPU-only ARM deployment with $\leq 1\%$ nDCG@10 loss.
(7)	Temperature softmax $P_T(t_i \mid t_{<i}) = \exp(z_i/T) / \sum_j \exp(z_j/T)$	Near-greedy decoding ($T = 0.1$) favours verbatim Pasal quotation over creative paraphrase.

Eq.	Name and compact form	Role in Patih
(8)	Precision / recall / F_1 $P = M \cap G / M , \quad R = M \cap G / G , \quad F_1 = 2PR / (P + R)$	Per-question citation accuracy. M = model citations, G = ground truth.
(9)	Micro-recall $R_{\text{micro}} = \sum_i M_i \cap G_i / \sum_i G_i $	Headline ‘‘Citation Accuracy’’ metric in the v6 evaluation (78.7 %).
(10)	Mean Jaccard $J = \frac{1}{N} \sum_i M_i \cap G_i / M_i \cup G_i $	Complementary set-overlap diagnostic (v6: 0.823).
(11)	Refusal precision $P_{\text{refusal}} = \text{TP} / (\text{TP} + \text{FP})$	First-class metric for calibrated abstention on Tier-4/5 questions (v6: 100 %, 10/10).
(12)	Entity Grounding $\text{EG} = \mathcal{E}(r) \cap \mathcal{E}(\text{ctx}) / \mathcal{E}(r) $	HalluGraph Layer-2: response entities must be a subset of context entities.
(13)	Relation Preservation $\text{RP} = \mathcal{R}(r) \cap \mathcal{R}(\text{ctx}) / \mathcal{R}(r) $	HalluGraph Layer-2: response relations must be supported by context.
(14)	AUC of a discriminator $\text{AUC} = \Pr[\text{score}(x^+) > \text{score}(x^-)]$	How well EG/RP separate faithful from hallucinated responses (HalluGraph: 0.979 on structured legal docs).
(15)	Defence-in-depth composition $p_L = p_0 \prod_{\ell=1}^L (1 - c_\ell)$	Multiplies catch rates of independent validator layers to bound residual hallucination $\leq 5\%$.
(16)	POMDP tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma, H)$	Mishra et al.’s formalisation of Agentic RAG; horizon $H=3$ in Patih’s Phase 3 plan.
(17)	POMDP expected return $J(\pi) = \mathbb{E}_\pi \sum_{t=0}^H \gamma^t R(s_t, a_t)$	Objective for any agentic policy; informs reward design at the loop boundary.
(18)	RAGAS Faithfulness $F = \{c \in \mathcal{C}(a) : c \models \text{ctx}\} / \mathcal{C}(a) $	Fraction of atomic claims in the answer entailed by the retrieved context.
(19)	RAGAS Answer Relevancy $\text{AR} = \frac{1}{m} \sum_j \cos(\phi(q), \phi(q^{(j)}))$	Cosine alignment between the original question and synthetic questions generated from the answer.
(20)	RAGAS Context Precision $\text{CP} = \frac{1}{ R^+ } \sum_k \text{rel}_k \sum_{j \leq k} \text{rel}_j / k$	Ordering quality of relevant chunks in the retrieved list.
(21)	Aggregated provider capacity $\Lambda = \sum_p \Lambda_p \approx 70\text{--}90 \text{ RPM}$	70–90 RPM headroom; justifies the four-tier free-tier fallback chain.
(22)	Steady-state arrival rate $\lambda = UQ / (60T_w) \approx 2.1 \text{ RPM}$	2.1 RPM expected from 100 daily active users over 8 work hours.
(23)	Little’s law $L = \lambda W$	Underpins the queue-length-versus-latency analysis.
(24)	Burst wait approximation $W_{\text{burst}} \approx B / \Lambda - \tau \text{ if } B / \tau > \Lambda$	Determines when the system breaks the P95 latency budget of 30 s.

Eq.	Name and compact form	Role in Patih
(25)	Token bucket update $b_{t+\Delta t} = \min(C, b_t + \Delta t \cdot r) - 1$ on request	Pre-emptive rate limiting in <code>app/llm/rate_limiter.py</code> ; capacity C , refill rate r .
(26)	Semantic-cache decision $\hat{a} = a^*$ if $\cos(\phi(q), \phi(q^*)) \geq \tau_{\text{cache}}$	Phase 2 GPTCache: serve answer a^* of a cached query q^* if cosine similarity to current query q exceeds threshold $\tau_{\text{cache}} \approx 0.95$.

5. End-to-End Data Flow

5.1 Offline ingestion pipeline (run once per document)

- PDF text extraction.** PyMuPDF reads text-native PDFs; Tesseract OCR (with the Indonesian language pack `tesseract-ocr-ind`) is the fallback for scanned PDFs.
- Structure parsing.** Explicit regular expressions identify BAB, Bagian, Pasal, ayat and huruf. The resulting tree is an Abstract Syntax Tree (AST) preserved at `data/parsed/permensos-8-2023.json`.
- Parent-Document chunking.** *Child* = one ayat, or one huruf if the ayat exceeds approximately 300 tokens. *Parent* = the entire Pasal. Children carry a `parent_id` pointer back to the Pasal chunk.
- Embedding.** Each child is passed through `multilingual-e5-large` quantised to INT8 in ONNX format and stored in a Chroma vector store backed by SQLite. (On disk-constrained machines, the Gemini embedding API substitutes for the local e5; the upgrade path is documented in `deploy/scripts/quantize_e5.py`.)
- Sparse indexing.** The same children are tokenised and indexed with `rank_bm25` into a pickled in-memory store at `data/bm25/`.

5.2 Online query pipeline (per user request)

- Language detection.** `lingua-py` with a confidence threshold of 0.7 classifies the query as Indonesian, English, or (default) Indonesian when ambiguous.
- Query translation (English path only).** The query is translated to Indonesian via Gemini Flash before embedding, leaving the corpus untouched (Section 3.11).
- Dense retrieval.** The query is embedded with the same model used at index time and queried against Chroma; top-15 children are returned.
- Sparse retrieval.** BM25 returns its own top-15.
- Reciprocal Rank Fusion.** The two ranked lists are merged into top-12 fused children (`top_k_fused = 12`; raised from the original 8 after Phase 1 tuning).
- Parent expansion.** Each child is replaced by its parent Pasal; duplicates are collapsed.
- Cross-reference resolution.** A regex on the parent text detects in-text references to other articles and pulls those Pasal into context.
- Always-on injection.** Pasal 1 (the *Ketentuan Umum*: definitions) is always prepended.

9. **LLM generation through LiteLLM.** The prompt-template engine fills a strict system prompt and the context, and dispatches to the fallback chain (Section 6.3).
10. **Validation.** The raw response is scored by the four-layer validator (Section 6.4).
11. **Display.** The validated answer, the confidence badge and the citation cards are streamed to the Chainlit UI.

6. Layer-by-Layer Architecture

Patih is decomposed into seven layers with isolated responsibilities. The separation is not cosmetic: it lets Phase 2 (multi-document and KG augmentation) *extend* rather than rewrite, lets evaluation isolate bottlenecks per layer, and localises failure modes (if the LLM provider chain is exhausted, retrieval still functions).

6.1 Layer 1 – Ingestion

Code: `app/ingest/` (`pdf_loader.py`, `structure_parser.py`, `chunker.py`, `doc_registry.py`, `validators.py`, `cli.py`).

Task. Convert a raw PDF into a structured AST and into chunks with full metadata.

Why a generic splitter fails. Indonesian regulatory PDFs are strongly hierarchical (BAB → Bagian → Pasal → ayat → huruf). A naive `RecursiveCharacterTextSplitter` (as shipped with LangChain) breaks chunks across Pasal boundaries, which is fatal for citation enforcement downstream. The structure-aware approach is consistent with Reuter et al. (2025) and with Yu & Tsai (2025)’s tree-based retrieval on Taiwanese regulations.

Strategy.

1. Extract text with PyMuPDF; fall back to Tesseract OCR if a page contains no extractable glyphs.
2. Parse with explicit regex; e.g.

```
r"Pasal\s+(\d+)\s*\n(.*) (?=Pasal\s+\d+|\Z) "
r"\((\d+)\)\s*(.*) (?=\(\d+)\|\Z) "
r"([a-z])\.\s*(.*) (?=[a-z]\.\|\Z) "
```

3. Validate the parse: compare the article count against the PDF table of contents. If the miss-rate exceeds 5%, the regex is refined.
4. Chunk: child = ayat (or huruf), parent = Pasal. Both are stored with metadata fields enumerated in `configs/chunking.yaml`.

6.2 Layer 2 – Retrieval

Code: `app/retrieval/` (`embedder.py`, `vector_store.py`, `bm25_store.py`, `hybrid.py`, `parent_expander.py`, `cross_ref_resolver.py`, `always_on.py`, `pipeline.py`).

The retrieval layer realises Sections 3.2–3.4 and Section 5. Key parameters (Phase 1 tuned):

- `top_k_dense = 15`, `top_k_sparse = 15`, `top_k_fused = 12`.
- RRF constant $k = 60$.
- Always-on injection of Pasal 1.
- Cross-reference depth: one hop (a referenced Pasal is fetched, but its referents are not).

6.3 Layer 3 – LLM Gateway

Code: `app/llm/ (gateway.py, generator.py, lang_detect.py, translator.py, rate_limiter.py, prompts.py)`.

Task. Generate an answer from (query, context) with resilience against free-tier rate limits.

Fallback chain. Each free-tier provider has its own ceiling:

Provider	Model	Free-tier limits (Phase 1 observed)
Google AI Studio	Gemini 2.5 Flash	15 RPM, ~20 RPD, 1 M TPM
Groq	Llama 3.3 70B	30 RPM, 14 400 RPD, 100 K TPD
Cerebras	Qwen 3 32B / Llama 3.1 8B	~30 RPM, 1 M tokens/day
OpenRouter	DeepSeek R1 (free)	variable, ~20 RPM, revocable

The router is LiteLLM (Section 3.13); a proactive token-bucket throttle avoids hard limits.

Prompt template (Indonesian system prompt). Strict rules in `configs/prompts/system_id.md`:

1. Answer *only* from the article excerpts in `<konteks>`.
2. Always emit a reference of the form (Pasal X ayat (Y) huruf z).
3. If the information is not in `<konteks>`, respond verbatim with the calibrated-abstention sentence “*Informasi tersebut tidak diatur secara spesifik dalam ...*” (per Zhang et al., 2025, Section 3.7).
4. Never invent a Pasal number.
5. For *definition* questions, always cite *Pasal 1 angka N*.
6. For *thematic / cross-reference* questions, cite each contributing Pasal separately rather than an umbrella article (Phase-1 tuning artefact, Section 10).
7. For category-specific keywords (*kewenangan, koordinasi, evaluasi*), prioritise the Pasal whose own header matches the category.

6.4 Layer 4 – Validators (Hallucination Defence in Depth)

Code: `app/validators/ (citation_extractor.py, whitelist_validator.py, entity_grounding.py, relation_preservation.py, pipeline.py)`.

Realises Section 3.8 (HalluGraph EG/RP, ensemble per Sardana 2025, VeriCite-style verification per Qian 2025). Four independent scorers:

1. **Citation extractor.** A regex parses every (Pasal X ayat (Y) huruf z) token in the LLM response.
2. **Whitelist validator.** Each extracted citation is looked up in the document registry. A citation to Pasal 99, when the document has only Pasal 1–34, is flagged as invalid.
3. **Entity Grounding (EG) – HalluGraph-inspired.** Entities in the response (institutions, defined legal terms) must be a subset of entities in the retrieved context. Phase 1 uses a curated legal-term list (`LEGAL_TERMS_PHASE1`); Phase 2 will upgrade to a proper Indonesian-legal NER.

4. Relation Preservation (RP) – HalluGraph-inspired. Relations between entities in the response (“X performs Y”) must be supported in the context. Phase 1 implements this with regex-driven content-word overlap, including a citation-only-line skipper (`_is_citation_only_line()`) that prevents footer / bullet citation lines from polluting the RP signal.

The validators emit a structured object with fields `citations_valid`, `eg_score`, `rp_score`, `confidence`, `hitl_flag`, and `hitl_reasons`. Flagged answers are routed to the HITL queue.

6.5 Layer 5 – Conversational UI

Code: `app/ui/` (`chainlit_app.py`, `components.py`, `history.py`).

Chainlit was preferred over Streamlit and Gradio because it provides native streaming responses, built-in source citation cards, conversation history persistence with feedback buttons (thumbs up/down) usable as HITL training signal, and a sidebar that hosts the welcome message and per-answer citation-card list. Components:

- `build_confidence_badge()` renders a green / yellow / red confidence chip from the validator output.
- `build_citation_cards()` performs a lazy-cached lookup of parent Pasal text (capped at 8 articles per answer).
- `ConversationStore` persists turns and feedback to `data/conversations.db` (SQLite).

6.6 Layer 6 – Evaluation

Code: `app/eval/`. Theory grounding in Section 3.12 (RAGAS + ensemble + calibrated abstention).

Test set (50 questions) – composition chosen to exercise distinct failure modes (consistent with the 5-tier eval design of brief v2):

Tier	Type	Count
T1	Factual / single-Pasal extraction	25
T2	Cross-reference (multi-Pasal)	10
T3	Thematic (concept across articles)	5
T4	Out-of-scope refusal	8
T5	Adversarial / definition trap	2

Metrics and thresholds.

- RAGAS *Faithfulness* ≥ 0.85 .
- RAGAS *Answer Relevancy* ≥ 0.85 .
- Custom *Citation Accuracy* $\geq 90\%$ – micro-recall against ground-truth Pasal list (precision tracked separately per Qian et al., 2024).
- *Refusal Precision* $\geq 80\%$ on T4+T5 (first-class metric per Zhang et al., 2025).
- P95 end-to-end *latency* ≤ 30 s.
- *Hard-failure rate* $\leq 5\%$.

Judge separation. `-provider` (generator) and `-judge-provider` (RAGAS judge) are independently configurable to avoid double-burning quotas.

6.7 Layer 7 – Observability

Code: `app/infra/`. Langfuse self-hosted on the same VM traces every request end-to-end: query → retrieval timing → LLM call (each fallback attempt logged) → validator scores → user feedback. Cost tracking is present even though it is currently zero, so a future migration to paid tiers comes with historical baselines.

7. Bilingual Indonesian–English Handling

Algorithmic restatement of Section 3.11:

1. Detect the input language with `lingua-py` (default ID on ambiguity).
2. If ID: embed query as-is, retrieve.
3. If EN: translate query → ID via Gemini Flash, embed the Indonesian rewrite, retrieve from the Indonesian corpus.
4. Generate the answer in the *original* input language; *never* translate the corpus.
5. Preserve Pasal citations verbatim in Indonesian; for English answers, format as “Article 5 paragraph (2) states: ‘... [Indonesian text]’. In English, this means: ‘...’”.
6. Regex post-validate that every **Pasal X ayat (Y)** reference in the response is followed by Indonesian-language text matching the retrieved context.

8. Deployment Topology

Patih is deployed as a *single-node* stack on an Oracle Cloud Always Free VM (ARM Ampere A1, 4 OCPU, 24 GB RAM, 200 GB block storage) running Ubuntu 24.04. The full stack runs under Docker Compose:

- **Chainlit** (chat UI) on port 8000.
- **Langfuse** (observability) on port 3000, backed by PostgreSQL on port 5432.
- **Application process** hosting the ingestion / retrieval / LLM / validator modules, with the e5-large ONNX INT8 embedder in-process, Chroma at `data/chroma/`, BM25 at `data/bm25/`, and LiteLLM dispatching to external free-tier APIs.
- **Public ingress** via Cloudflare Tunnel (free, no inbound ports opened on the VM, built-in DDoS protection).
- **Backups** via cron: a nightly tar of `data/chroma/` and `data/bm25/` is uploaded to OCI Object Storage (20 GB Always Free tier).

Single-node is justified by the load: ~ 100 active users per day $\approx 1500\text{--}5000$ queries per day $\approx 0.02\text{--}0.06$ queries per second on average; multi-node would be over-engineering.

9. Phase 1 Build Order (chronological)

1. **Step 1 – Scaffolding.** Poetry project, configs, base test harness.
2. **Step 2 – Ingestion.** PDF loader, structure parser, parent-document chunker, document registry; validated against the human-counted Pasal list of Permensos 8/2023.

3. **Step 3 – Indexing.** Embedder, Chroma vector store, BM25 pickle, hybrid + RRF, parent expander, cross-reference resolver, always-on Pasal 1 injector.
4. **Step 4 – LLM gateway.** LiteLLM router, four-tier fallback chain, rate-limit token bucket, prompt templates (ID, EN, bilingual, few-shot), language detector, translator.
5. **Step 5 – Validators.** Citation extractor, whitelist validator, EG, RP, full pipeline aggregator. Adversarial test: a fabricated “Pasal 99” is caught by three independent reasons (`invalid_citation`, `low_eg`, `low_rp`). 190 tests total, 183 pass.
6. **Step 6 – UI.** Chainlit entry point, welcome message + disclaimer, citation cards, confidence badge, thumbs feedback, SQLite conversation history.
7. **Step 7 – Evaluation.** 50-question test set; eval CLI with subset and endurance subcommands; five baselines (v1–v6).
8. **Step 8 – Deployment (pending).**
9. **Step 9 – Endurance test (pending).**
10. **Step 10 – Final acceptance review (pending).**

10. Evaluation Trajectory (v1–v6)

Five real evaluation runs were performed as Phase 1 progressed. Each run produced a measurable shift, traced to a specific code change:

Metric	Threshold	v3	v4	v5	v6
RP mean (answered)	≥ 0.80	0.67	0.91	0.9833	0.98
EG mean (answered)	≥ 0.90	1.00	1.00	0.9975	1.00
Refusal precision	$\geq 80\%$	100%	100%	100%	100%
P95 latency	≤ 30 s	4.5 s	7.7 s	4.8 s	4.3 s
Hard-failure rate	$\leq 5\%$	19/50	13/50	0/50	0/50
Citation accuracy	$\geq 90\%$	76.9%	81.8%	75.4%	78.7%
RAGAS Faithfulness	≥ 0.85	nan	nan	nan	nan
RAGAS Answer Relevancy	≥ 0.85	nan	nan	nan	nan

What each run taught us.

- *v3*. Quota cascade exposed (Gemini 20 RPD, Groq 100 K TPD), and a RAGAS context-bug (passing chunk numbers instead of chunk text) returned NaN for Faithfulness / Answer Relevancy. Fix: throttle, separate `parent_chunks` text into `GenerationResult`.
- *v4*. `_is_citation_only_line()` added to RP – skips Llama’s bulleted footer citations – RP jumped 0.67 $\rightarrow$ 0.91, validating the diagnosis. Judge-provider split introduced (`-judge-provider`).
- *v5*. First clean run: 0/50 hard failures with Groq Llama 3.3 70B handling all 50 questions end to end. Citation accuracy 75.4% diagnosed as model-side limit, not retrieval.
- *v6*. Three new system-prompt rules (definition, thematic, category-specific) plus `top_k_fused = 12`. Net +3.3 pp on micro-recall; T1 regressed slightly while T2/T3 improved. Tuning saturation reached – further headroom requires either a stronger model, per-*angka* chunking of Pasal 1, or test-set curation.

11. Roadmap

Phase 2 – Multi-document and Knowledge-Graph augmentation

- Ingest the remaining 27 PDFs (Permensos, UU 21/2007, UU 18/2017, PP, Perbup).
- Adopt *Summary-Augmented Chunking* (Reuter et al., 2025) to mitigate DRM (Section 3.4).
- Build a Knowledge Graph of *Pasal-ayat-references*, plus inter-regulation links, per Goel et al. (2026) (Section 3.10).
- Adopt a GraphRAG-style retrieval pattern for thematic queries (Edge et al., 2024).
- Migrate vector DB to Qdrant self-host or pgvector once filtered ANN becomes necessary (Pan et al., 2023; Amanbayev et al., 2026).
- Add a query-complexity classifier and adaptive parameter tuning per HyPA-RAG (Kalra et al., 2025).
- Extend the eval set to 100 questions including cross-document reasoning.
- Trigger migration to a paid tier (Gemini Flash paid \$5–15/month) once free-tier RPM is exhausted.

Phase 3 – Agentic + production HITL

- Multi-step reasoning planner combining Self-RAG (Asai et al., 2023), CRAG (Yan et al., 2024), Doctor-RAG (Jiao et al., 2026) and Chain-of-Retrieval (Wang et al., 2025), bounded to max 3 hops and guarded by the SoK cautions of Mishra et al. (2026).
- Replace the regex EG/RP with an NLI-based verifier (VeriCite, Qian et al., 2025; “Fast and Faithful”, Liu et al., 2026).
- Cross-lingual upgrade via CroSearch-R1 (Qi et al., 2026) if bilingual usage scales.
- A full HITL reviewer dashboard.
- Compliance audit trail (Indonesian Personal Data Protection / UU 27/2022, log retention).
- Possible migration to Claude Haiku 4.5 / Sonnet 4.6 routing (\$30–100/month).

12. Trade-offs Explicitly Accepted in Phase 1

Trade-off	Phase 1 choice	Rejected alternative	Reason
Citation discipline	Gemini Flash + validators	Claude Sonnet 4.6	Sonnet is paid.
Multi-document reasoning	Single document	GraphRAG immediately	Premature; Goel 2026 cost not justified yet.
Reranker	RRF only	Cohere Rerank API	Cohere is paid; RRF + parent expansion sufficient.

Trade-off	Phase 1 choice	Rejected alternative	Reason
Embedder hosting	e5-large ONNX self-host (target) / Gemini embedding (current)	Paid embedding API	Free tier is rate-limited; self-host viable on ARM CPU.
Agentic loop	None in Phase 1	Self-RAG / CRAG from start	Mishra 2026 SoK cautions on compounding hallucination.
HITL	JSONL flag queue	Full reviewer dashboard	Personal MVP; Phase 2 work.
Auth	None (single-tenant)	OAuth multi-tenant	Personal/exploratory MVP.
Latency budget	P95 ≤ 30 s	Sub-second	Free-tier rate limits force queueing.

13. Open Decision Points

1. **Oracle region.** Ampere A1 capacity is often unavailable in Singapore / Tokyo; Hyderabad / Osaka should be checked at provisioning time.
2. **Canonical PDF.** Two copies of Permensos 8/2023 exist at the project root; they must be hash-compared and the duplicate removed.
3. **KG technology for Phase 2.** Neo4j (graph-native query semantics) versus Postgres-as-KG (familiar tooling, free tier on Supabase).
4. **Self-hosted LLM for data-residency cases.** A Sahabat-AI 70B or Qwen 3 32B self-host needs a GPU budget (\$50–200/month for H100 rental); deferred to Phase 3.

14. Repository Layout

app/ingest/	PDF load, structure parse, chunk, doc registry
app/retrieval/	Dense + BM25 hybrid, RRF, parent expand, cross-ref
app/llm/	LiteLLM gateway, prompts, lang detect, generator
app/validators/	Citation extractor, whitelist, EG, RP
app/eval/	RAGAS runner, custom metrics, latency, report
app/ui/	Chainlit app, citation cards, conversation history
app/infra/	Langfuse tracer, healthcheck
configs/	litellm.yaml, chunking.yaml, prompt templates
deploy/	Docker Compose, Dockerfile, systemd units, scripts
tests/	unit/, integration/, golden/
data/	raw/, parsed/, chroma/, bm25/, registry/, test/
docs/	this document and related notes

15. Disclaimer

Patih is a regulatory-research assistant, **not legal advice**. Every reference must be verified against the original regulation before any operational or legal decision is taken.

A. Annotated Bibliography

Every source below shaped a concrete decision in Patih. For each, we state the citation, the headline claim, and how Patih operationalised it. Entries are grouped by the theoretical section that uses them.

A.1 Retrieval architecture

Butler & Butler (2026), *Legal RAG Bench: an end-to-end benchmark for legal RAG* (arXiv:2603.01710).

100 hand-crafted questions over the Victorian Criminal Charge Book. Hierarchical error decomposition shows embedder swap gives +17.5 pp correctness vs. moderate LLM-swap effect; many “hallucinations” are retrieval failures. **Used by Patih:** Section 3.2; motivates concentrating engineering effort on the retrieval layer first.

Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts* (arXiv:2307.03172).

Documents the U-shaped recall curve in long-context LLMs: information in the middle is recalled significantly worse than at the start or end. **Used by Patih:** Section 3.1; the reason Patih chunks instead of stuffing the full document into context.

Leng et al. (2024), *Long Context RAG Performance of Large Language Models* (arXiv:2411.03538, Databricks).

20 LLMs, contexts from 2k to 2M tokens. Only the most recent SOTA models sustain accuracy above 64k; documents distinct failure modes. **Used by Patih:** Section 3.1; the empirical companion to Lost-in-the-Middle.

Reuter et al. (2025), *Towards Reliable Retrieval in RAG Systems for Large Legal Datasets* (arXiv:2510.06999).

Names *Document-Level Retrieval Mismatch* (DRM) and proposes *Summary-Augmented Chunking* (SAC). Surprising finding: generic summaries beat domain-expert summaries. **Used by Patih:** Section 3.4; SAC is on the Phase 2 roadmap.

Yu & Tsai (2025), *Tree-Based Text Retrieval via Hierarchical Clustering in RAG Frameworks: Application on Taiwanese Regulations* (arXiv:2506.13607).

Hierarchical-clustering retrieval that adaptively picks granularity without a fixed k ; outperforms flat top- k on Taiwanese legal corpora. **Used by Patih:** Section 3.4; nearest published analog of Patih’s Pasa-ayat-huruf chunking.

Cormack, Clarke & Büttcher (2009), *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods* (SIGIR’09).

Original RRF paper with $k = 60$ default. **Used by Patih:** Section 3.3; the fusion formula in `app/retrieval/hybrid.py`.

Kalra et al. (2025), *HyPA-RAG: A Hybrid Parameter-Adaptive Retrieval-Augmented Generation System for AI Legal and Policy Applications* (arXiv:2409.09046).

NYC Local Law 144 case study. Query-complexity classifier + adaptive parameter tuning + dense+sparse+KG hybrid. **Used by Patih:** Section 3.3; Phase 2 reference design.

A.2 Embedder and language-model selection

LazarusNLP community (2025), *Indonesian Sentence Embeddings benchmark* (GitHub).

Community-maintained leaderboard for Indonesian retrieval. `multilingual-e5-large` leads MIRACL-id at R@1 70.83 % / nDCG@10 76.16 %. **Used by Patih:** Section 3.5; basis of the embedder choice.

Chen et al. (2024), *BGE-M3: Multi-Functionality, Multi-Linguality, Multi-Granularity Text Embeddings* (arXiv:2402.03216).

Unified dense + sparse + multi-vector in one model. **Used by Patih:** Section 3.5; considered and rejected as the embedder choice for Phase 1.

Koto et al. (2024), *Cracking the Code: Multi-domain LLM Evaluation on Real-World Professional Exams in Indonesia* (arXiv:2409.08564).

27 LLMs tested on Indonesian professional exams. Indonesian-targeted small models (Merak, Komodo, Cendol, SEA-LION old) perform near-random on legal reasoning. **Used by Patih:** Section 3.6; the reason Patih does not default to small Indonesian models.

Susanto et al. (2025), *SEA-HELM: Southeast-Asian Holistic Evaluation of Language Models* (arXiv).

SEA-region LLM leaderboard. Qwen 3 family consistently leads. **Used by Patih:** Section 3.6; underpins choice of Qwen on Cerebras as a fallback tier.

A.3 Hallucination defence and citation discipline

Magesh et al. (2024), *Hallucination-Free? Assessing the Reliability of Leading AI Legal Research Tools* (Stanford HAI / Journal of Empirical Legal Studies).

Westlaw 34 % hallucination, Lexis+ 17 %, Practical Law / Ask GPT 17 %. Falsifies the “hallucination-free” marketing claim. **Used by Patih:** Section 3.6, Section 3.8; sets the realistic upper bound on what Patih can achieve.

Noël et al. (2025), *HalluGraph: Auditable Hallucination Detection for Legal RAG Systems via Knowledge Graph Alignment* (arXiv:2512.01659).

KG-aligned detector: Entity Grounding + Relation Preservation. AUC = 0.979 on structured legal docs; beats semantic-similarity baselines because the latter tolerate entity substitution. **Used by Patih:** Section 3.8; directly inspired app/validators/entity_grounding.py and relation_preservation.py.

Qian et al. (2024), *On the Capacity of Citation Generation by Large Language Models* (arXiv:2410.11217).

Citation metrics over-penalise unnecessary citations; proposes *Generate-then-Refine* to add missing relevant citations and prune irrelevant ones without changing answer text. **Used by Patih:** Section 3.7; Patih separates micro-recall and precision in evaluation.

Qian et al. (2025), *VeriCite: Towards Reliable Citations in Retrieval-Augmented Generation via Rigorous Verification* (SIGIR-AP 2025; arXiv:2510.11394).

Three-stage pipeline: initial answer → NLI claim verification → evidence selection → refinement. **Used by Patih:** Section 3.7; Patih’s Layer 2 is the simplified regex-and-set variant; full NLI is Phase 3.

Zhang, Gray, Savelka & Ashley (2025), *Measuring Faithfulness and Abstention: An Automated Pipeline for Evaluating LLM-Generated 3-ply Case-Based Legal Arguments* (arXiv:2506.00694).

Defines faithfulness, factor utilisation and abstention as three independent metrics. Most LLMs fail the abstention test even when instructed to stop. **Used by Patih:** Section 3.7; refusal precision is a first-class metric in Tier 4/5 of the eval set.

Sardana (2025), *Real-Time Evaluation Models for RAG: Who Detects Hallucinations Best?* (arXiv:2503.21157).

Benchmarks LLM-as-Judge, Prometheus, Lynx, HHEM, TLM. No single winner; ensembles are more reliable. **Used by Patih:** Section 3.8; Patih’s four validators are a heterogeneous ensemble.

Liu et al. (2026), *Fast and Faithful: Real-Time Verification for Long-Document RAG Systems* (arXiv:2603.23508).

Production-scale real-time verifier with adaptive inference. **Used by Patih:** Section 3.8; Phase 3 candidate for replacing the regex EG/RP.

A.4 Agentic and GraphRAG (Phase 2 / Phase 3)

Asai et al. (2023), *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection* (arXiv:2310.11511).

Self-reflection tokens governing when/what to retrieve. **Used by Patih:** Section 3.9; Phase 3 candidate pattern.

Yan et al. (2024), *Corrective RAG*, with open-source reproduction by Yalavarthi (2026, arXiv:2603.16169).

Retrieval evaluator gates a corrective web-search fallback. The reproduction’s SHAP analysis shows the evaluator works primarily on named-entity alignment. **Used by Patih:** Section 3.9; Phase 3 candidate; the entity-alignment insight reinforces HalluGraph EG.

Jiao et al. (2026), *Doctor-RAG: Failure-Aware Repair for Agentic Retrieval-Augmented Generation* (arXiv:2604.00865).

Diagnose-and-repair framework that localises failure points and reuses validated reasoning prefixes. **Used by Patih:** Section 3.9; Phase 3 plan for failure-aware retries.

Wang et al. (2025), *Chain-of-Retrieval Augmented Generation* (arXiv:2501.14342).

o1-style RAG with dynamic query reformulation; > 10 EM-point gain on multi-hop QA. **Used by Patih:** Section 3.9; Phase 3 multi-hop pattern.

Mishra et al. (2026), *SoK: Agentic Retrieval-Augmented Generation: Taxonomy, Architectures, Evaluation, and Research Directions* (arXiv:2603.07379).

Formalises agentic RAG as a finite-horizon POMDP; catalogues compounding hallucination propagation, memory poisoning, retrieval misalignment, cascading tool-execution vulnerabilities. **Used by Patih:** Section 3.9; the explicit reason Patih has *no* agentic loop in Phase 1.

Edge et al. (2024 / 2025), *From Local to Global: A Graph-RAG Approach to Query-Focused Summarization* (arXiv:2404.16130, Microsoft).

LLM-driven entity extraction → community detection → map-reduce QA at the 1 M-token scale. **Used by Patih:** Section 3.10; Phase 2 thematic-query pattern.

Goel et al. (2026), *Domain-Partitioned Hybrid RAG for Legal Reasoning: Toward Modular and Explainable Legal AI for India* (arXiv:2602.23371).

Three RAG modules + Neo4j legal KG + agentic orchestrator. 70% pass rate vs. 37.5% RAG-only baseline. **Used by Patih:** Section 3.10; the closest published analog and Patih’s Phase 2 architectural template.

A.5 Evaluation, observability and production engineering

Es et al. (2023), *RAGAS: Automated Evaluation of Retrieval Augmented Generation* (arXiv:2309.15217).

Reference-free metrics: faithfulness, answer relevancy, context precision/recall. **Used by Patih:** Section 3.12; the framework behind app/eval/.

Pan, Wang & Li (2023), *Survey of Vector Database Management Systems* (arXiv:2310.14021).

Five obstacles to vector data management; taxonomy of native vs. extended VDBMSs. **Used**

by Patih: Section 3.13; informs the Phase 1 Chroma choice and the Phase 2 Qdrant / pgvector plan.

Amanbayev et al. (2026), *Filtered Approximate Nearest-Neighbour Search in Vector Databases: System Design*.

Pre-filter vs. post-filter trade-offs for filtered ANN. **Used by Patih:** Section 3.13; per-regulation filtered ANN is part of the Phase 2 plan.

Tian Pan (2026), blog: *Tail-tolerant retry policy for LLM gateway latency cliffs*.

Default LiteLLM retry policies add tail latency; tune timeouts aggressively. **Used by Patih:** Section 3.13; sets the 5 s-per-attempt / 15 s-budget retry config.

A.6 Cross-lingual retrieval

Badenes-Olmedo et al. (2019), *Legal document retrieval across languages: topic hierarchies based on synsets*.

Cross-lingual legal IR via EuroVoc synsets. **Used by Patih:** Section 3.11; historical baseline supporting “translate the query, not the corpus”.

Qi et al. (2026), *CroSearch-R1: Better Leveraging Cross-lingual Knowledge for Retrieval-Augmented Generation* (arXiv:2604.25182).

Search-augmented RL with multilingual rollouts and cross-lingual knowledge integration via GRPO. **Used by Patih:** Section 3.11; Phase 3 cross-lingual upgrade candidate.

A.7 Background and adjacent benchmarks

These were consulted while shaping decisions but are not citation-load-bearing today:

- Paul et al. (2023), *Pre-trained Language Models for the Legal Domain: A Case Study on Indian Law*.
- Joshi et al. (2024), *IL-TUR: Benchmark for Indian Legal Text Understanding and Reasoning*.
- Öztürk et al. (2026), *HUKUKBERT: Domain-Specific Language Model for Turkish Law*.
- Baba Ahmadi et al. (2026), *LEMUR: A Corpus for Robust Fine-Tuning of Multilingual Law Embedding Models*.
- Han et al. (2025), *Retrieval-Augmented Generation with Graphs (GraphRAG survey)*.
- Ni et al. (2025), *StepChain GraphRAG*.
- Fan et al. (2026), *Do We Still Need GraphRAG? – benchmarking RAG vs. GraphRAG*.
- Keisha et al. (2025), *All for Law and Law for All: Adaptive RAG Pipeline for Legal Research*.
- Shanivendra (2026), *Citation-Enforced RAG for Fiscal Document Intelligence*.
- Lelong et al. (2026), *Agentic RAG with Knowledge Graphs for Complex Multi-Hop Reasoning*.
- Ockerman et al. (2025), *Exploring Distributed Vector Databases on HPC Platforms*.

A.8 Patih internal documents

- Build-spec Phase 1 zero-budget: `D:\Research\Project Data\k1\research\chatbot-permensos-tpo\dra`
- Brief v1 (single-doc baseline): `brief-tech-stack.md`
- Brief v2 (multi-doc legal reasoning): `brief-v2-legal-reasoning.md`
- Zero-budget addendum: `brief-v2-addendum-zero-budget.md`
- Decisions log: `decisions.md`
- Chat transcript: `docs/patih.txt`